

Network Namespaces in Unicon

Jafar Al-Gharaibeh

Unicon Technical Report: 30

July 2026

Abstract

Linux network namespaces isolate a process's network stack – interfaces, addresses, and routes – without a full container. This report describes native support in Unicon: `open()` mode "j" creates a namespace handle; an optional leading handle on `fork()`, `spawn()`, and `system()` runs the child inside that namespace; `ns["name"] / key(ns)` peek status. Trailing `persist=`, `bridge=`, and `usersn=` attributes cover `ip(8)` visibility, a pre-created bridge, and unprivileged bootstrap. Package `net` supplies link helpers (`veth`, `bridge`) on top of those handles.

Keywords: Unicon, network namespaces, Linux, `open`, `fork`, `spawn`, `system`, runtime, technical report.

Unicon Project
<http://unicon.org>
University of Idaho
Department of Computer Science
Moscow, ID, 83844, USA

1 1. Introduction

This report describes Linux network namespaces [15] in Unicon: how a handle is opened, how a child process or thread joins it, how status is peeked, and what remains out of scope. It is formatted as a Unicon Technical Report [1]. The language-facing reference lives in *Programming with Unicon* [7] (Chapter 5, "Network Namespaces"; Chapter 14, `unettopo`) and the language reference (`open mode j`, optional leading namespace file on `fork / spawn / system`, subscript `peek`, `key()`). Automated tests live in `tests/posix/netns.icn` and `tests/unicon/nettopo.icn`.

The feature is optional and Linux-only. A build with `unshare()` / `setns()` and `CLONE_NEWNET` reports `Network namespaces` in `&features` (the `_NETNS` flag). Other hosts omit the feature; mode `j` fails with error 1045.

2 2. Motivation

Test topologies on Linux are often a handful of network namespaces, veth pairs, and optional bridges: isolated stacks wired together, cheap enough to create per test run. Unicon already treats `open()` as the constructor for files, sockets, TLS, SSH, and crypto handles. Namespace membership is the same kind of resource: create it, pass it, close it. Shelling out to `ip netns` works, but it cannot join the Unicon process itself, and it cannot put `fork()` / `spawn()` / `system()` children into a namespace without a second `ip netns exec` wrapper.

3 3. Design principles

One OS thread is in one network namespace for its lifetime. Membership lives on the kernel `nsproxy`. `fork(ns)` and `system(ns, ...)` join in the child process; `spawn(ns, ...)` joins on the new OS thread before any Icon code runs. The runtime never migrates a live thread between namespaces.

Existing verbs, not new globals. Mode `j` and trailing `name=value` attributes follow SSL and SSH. `fork`, `spawn`, and `system` take an optional leading namespace file, the same role a window plays in `DrawPoint(w, x, y)`. Link configuration that `ip(8)` already expresses stays in package `net`.

Status is peeked; `Attrib()` does not set netns fields. `ns["name"]` and `key(ns)` match sockets, TLS, SSH, and crypto handles. Writes for interfaces, addresses, and routes are deferred; use package `net`.

Create needs privilege or a user namespace. `unshare(CLONE_NEWNET)` requires `CAP_NET_ADMIN`, or a successful unprivileged user-namespace bootstrap ([section 6](#)).

4 4. Opening and closing

Mode "j" creates a namespace named `s1`. The result is a file handle, not a read/write stream. Other mode letters combined with j are error 209.

```
procedure main()
  local ns
  ns := open("r1", "j") | stop(&errortext)
  write(image(ns))
  close(ns)
end
```

Output:

```
netns(r1)
```

Implementation: a short-lived child calls `unshare(CLONE_NEWNET)`, bind-mounts `/proc/self/ns/net` onto a private path under `/tmp/unicon-netns-<uid>/`, brings `lo` up, and exits. The private mount pins the kernel namespace. `Fs_NETNS` (040 when the feature is built) marks the handle; state is a `struct NetnsFile *` on the file block.

Trailing attributes are `name=value` strings. Booleans are exactly `yes` or `no`. Unknown names or other boolean spellings are error 205.

Attribute	Role
<code>persist=yes no</code>	Also bind-mount at <code>/var/run/netns/<name></code> so
<code>bridge=yes no</code>	Best-effort <code>bridge0</code> inside the new namespace.

Attribute	Role
<code>usersns=yes no</code>	Force or forbid the unprivileged bootstrap in

Ephemeral opens (no `persist`) have no user-visible `ip netns` name. Two ephemeral `open("r1", "j")` calls yield two independent objects. `persist=yes` uses `O_EXCL`; a second `persist` open of the same name fails.

`close(ns)` always unmounts this handle's pins. It does not wait on `ns["refcount"]`. Unmounting a pin does not destroy the kernel namespace: any process that already joined still holds an `nsproxy` reference. After `close()`, a `persist` name disappears from `ip netns list`, and a later `open()` of that name creates a new namespace. Live handles are also unmounted at process exit and on `SIGINT` / `SIGTERM` when those signals still have the default disposition. Cleanup runs only in the creating process, so `fork()` children that exit do not tear down the parent's pins. `SIGKILL` can still leave orphan mounts.

5 5. Joining a namespace

The namespace file leads when it is the scope of the call.

`fork(ns)` – POSIX `fork()` with one optional argument. In the child, `setns()` runs before the function returns, so the first Icon statement is already namespaced. `fork()` with no argument is unchanged. `exec()` is unchanged: a child that already joined does not need extra namespace awareness.

`spawn(ns, ce, ...)` – optional leading file, then the existing block/string/stack sizes. `spawn()` holds the namespace object, stores it on the co-expression, and starts the OS thread. The trampoline joins before any Icon code runs on that thread, then releases the hold, so `close(ns)` from the parent cannot race the join. With concurrent threads, the OS thread starts immediately; no `@ce` activation is required. `wait(ce)` waits for it to finish.

`system(ns, argv, ...)` – same leading-file convention. The child joins before `dup` / `exec`.

```
procedure main()
  local ns, pid
  ns := open("r1", "j") | stop(&errorText)
```

```

pid := fork(ns) | stop("fork failed: ", &errortext)
if pid = 0 then {
    system(["ip", "link", "show"])
    exit(0)
}
wait(pid)
system(ns, ["ip", "link", "show"])
close(ns)
end

```

`fork()` fails (no value) only on the system-call error. A successful child result is 0, so discriminate with comparison, not `else`.

6 6. Unprivileged bootstrap

Creating a network namespace needs `CAP_NET_ADMIN` in the current user namespace [16]. `usersns=yes` enters a new user namespace, maps the calling uid/gid to 0 inside it, enters a private mount namespace, and then creates the `netns`. The mapping is written by a short-lived `fork()` helper that stays in the host user namespace; writing `uid_map` from the same process that called `unshare(CLONE_NEWUSER)` is rejected on hosts that restrict unprivileged user namespaces.

If `usersns=` is omitted and the privileged create fails with `EPERM` or `EACCES`, `open(..., "j")` retries once after the same bootstrap. Explicit `usersns=yes` bootstraps up front. Explicit `usersns=no` never bootstraps.

`unshare(CLONE_NEWUSER)` fails with `EINVAL` if the process is already multithreaded. That must happen before any `spawn()`-created thread exists; the `&errortext` states the constraint.

After a successful bootstrap the Unicon process is uid 0 *inside* the user namespace for the rest of its life. Later `open(..., "j")`, `fork(ns)`, `spawn(ns, ...)`, `system(ns, ...)`, and `ip` via package `net` need no further host privilege. A private `tmpfs` is mounted on `/var/run/netns` so `persist=yes` can create entries when the host directory is not writable to a mapped-root user namespace.

Linux security modules may still deny further `CLONE_NEWNS` / `CLONE_NEWNET` unshares for unprivileged callers. In that case `open()` fails with the system `&errortext`; run with `CAP_NET_ADMIN`, or relax the host policy.

`ns["userns"]` reports whether this handle was created after bootstrap. The choice is fixed at `open()`.

7 7. Status peek

```
procedure main()
  local ns, k
  ns := open("r1", "j", "persist=yes") | stop(&errortext)
  write("name=", ns["name"])
  write("persist=",
ns["persist"] | "no")
  write("userns=",
ns["userns"] | "no")
  write("refcount=", ns["refcount"])
  every k := key(ns) do
    write("field ", k)
  close(ns)
end
```

Unknown names raise 1336 (bad `netns` status field). An unpopulated field fails. Boolean fields that answered succeed with `"yes"` or `&null`. `key(ns)` generates every answerable field. `ns["*"]` snapshots those fields under one lock. Peeking a closed handle is error 174. `key(ns)` that has not yet produced a name also raises 174 if the handle is already closed.

Name	Value
<code>name</code>	Namespace name from <code>open()</code>
<code>persist</code>	<code>"yes"</code> or <code>&null</code>
<code>userns</code>	<code>"yes"</code> or <code>&null</code>
<code>refcount</code>	Integer; the open file plus any <code>spawn()</code> hold

`Attrib()` has no `netns` setters. Interface, address, and route configuration lives in package `net`.

8 8. Link helpers

`uni/lib/netns.icn` is pure Unicon. It shells out to `ip / ip netns`. There is no `rtnetlink` binding.

`Netns(name)` prefers `native open(name, "j", "persist=yes")` and keeps the file open so the `persist` mount stays pinned. If that fails it tries an ephemeral `native open`, then `ip netns add`. `delete()` closes the file or runs `ip netns del`.

`Link` is the interface base. `Veth`, `Bridge`, `Vrf`, and `Tun` create devices; a bare `Link(name)` wraps an existing interface. Every mutating method goes through `run()`, which prefers `system(nsfile, argv)` on a native handle and otherwise uses `ip netns exec`.

`Link.move(target)` accepts a `Netns` instance, a string name, or a native file (`target["name"]`). `Veth.peer()` returns a host-resident `Link` for the other end and does not copy `ns`; the two ends usually move into different namespaces. Create a `veth` *inside* a namespace when running unprivileged: host `RTNETLINK` create is often denied after user-namespace bootstrap.

```
import net

procedure main()
  local a, b, v, peer
  a := Netns("r1") | stop(&errortext)
  b := Netns("r2") | stop(&errortext)
  v := Veth("v1", "v2", a) | stop(&errortext)
  peer := v.peer()
  peer.move(b) | stop(&errortext)
  v.addr("10.0.0.1/24")
  peer.addr("10.0.0.2/24")
  v.up()
  peer.up()
  system(a.nsfile, ["ping", "-c", "1", "10.0.0.2"])
  v.delete()
  a.delete()
  b.delete()
end
```

`uni/progs/unettopo.icn` is that topology as a demo.

9 9. Runtime implementation

Open / close. `src/runtime/fsys.r` parses mode `j` and the trailing attributes, then calls `netns_create()`. `close()` releases the `NetnsFile`.

Join. `src/runtime/fxposix.ri` (`fork`), `src/runtime/fsys.r` (`system`), and `src/runtime/rcoexpr.r` (`nctramp` for `spawn`) call `netns_join()`. `spawn()` in `fmisc.r` uses `OptNetns` (`grttin.h`), the same optional-leading-arg pattern as `OptWindow`.

Helpers. `src/common/rnetns.c` implements `create`, `join`, `hold/release`, `usersn` bootstrap, and the `exit/signal` reaper. `configure.ac` link-checks `unshare` / `setns` / `CLONE_NEWNET`. `Fs_NETNS` is 0 when the feature is off so `OptNetns` still compiles.

Peek. `netns_peek` in `src/runtime/rposix.r` uses the same `filepeek` table as `sockets` and `SSH`. `oref.r` implements `ns["name"]`; `fstruct.r` implements `key(ns)`. Unknown names are 1336.

Image. `image(ns)` is `netns(<name>)`.

CLOEXEC. No long-lived namespace `fd` is held; `pins` are paths. `netns_join` opens with `O_CLOEXEC`. `Create` and bootstrap pipes use `pipe2(O_CLOEXEC)`.

10 10. Status and remaining work

The language surface described here is implemented: mode `j`, `persist/bridge/usersn` attributes, `fork(ns)` / `spawn(ns, ...)` / `system(ns, ...)`, status `peek`, package `net` `veth/bridge` helpers, and the `unettopo` demo.

Native `rtnetlink`. Link helpers still shell out to `ip(8)`.

Other platforms. No BSD jail or Windows backend.

PID and mount isolation. `fork(ns)` joins only the network namespace.

`Attrib()` writes for interface, address, and route state. Use package `net`.

`Netns.list()` / `Netns.current()`, link statistics, and `macvlan/ipvlan` classes are not provided.

11 Appendix: test coverage

`tests/posix/netns.icn` drops to `nobody` when started as root, then opens with `usersn=yes`. It checks:

- feature string and a successful `open(..., "j")`
- `ns["name"]`, boolean `persist` / `usersns`, `refcount`, `key(ns)`, and `ns["*"]`
- `fork(ns)` and `system(ns, ...)` see only `lo`
- `spawn(ns, ...)` when concurrent threads are present
- `persist` listing in `ip netns list`, gone after `close()`
- ephemeral name reuse vs `persist O_EXCL`

Without a working `usersns` or `CAP_NET_ADMIN` it prints `This Program Requires .`
`.. so Makefile.test` does not treat the run as `FAIL`.

`tests/unicon/nettopo.icn` covers package `net`: two `Netns` objects, a `veth` created inside the first namespace, peer move, addresses, and `ip netns exec` `iface` lists.

Expected output is `tests/posix/stand/netns.std`:

```
feature=yes
open=ok name=ut-ns persist=no usersns=yes refcount=1
key=ok
fork-child: ifaces=lo
fork-parent: ok
system: ifaces=lo
spawn: ifaces=lo
close=ok
persist=ok name=ut-nsp persist=yes
bridge: ifaces=bridge0,lo
listed=yes
listed-after-close=no
dup-ephemeral=ok
dup-persist=eexist
done
```

12 References

References

- [1] Clinton Jeffery. "How to Write a Unicon Technical Report". doc/utr/utr15.tex. 2013.
- [2] The libssh Project. "libssh – The SSH Library". <https://www.libssh.org/>. 2026.
- [3] Clinton Jeffery. "The Unicon Messaging Facilities". doc/utr/utr13.odt.
- [4] The OpenSSL Project. "OpenSSL – Cryptography and SSL/TLS Toolkit". <https://www.openssl.org/>. 2026.
- [5] H. Krawczyk and M. Bellare and R. Canetti. "HMAC: Keyed-Hashing for Message Authentication". RFC 2104. 1997.
- [6] S. Deering. "Host Extensions for IP Multicasting". RFC 1112. 1989.
- [7] Clinton Jeffery and Shamim Mohamed and Jafar Al-Gharaibeh. "Programming with Unicon". doc/book/. 2026.
- [8] H. Holbrook and B. Cain. "Source-Specific Multicast for IP". RFC 4607. 2006.
- [9] Jafar Al-Gharaibeh. "Raw Sockets and Packet Layouts in Unicon". doc/utr/utr29.rst. 2026.
- [10] Jafar Al-Gharaibeh. "Multicast and Socket Attributes in Unicon". doc/utr/utr28.rst. 2026.
- [11] J. Postel. "Internet Control Message Protocol". RFC 792. 1981.
- [12] W. Fenner. "Internet Group Management Protocol, Version 2". RFC 2236. 1997.
- [13] B. Cain and S. Deering and I. Kouvelas and B. Fenner and A. Thyagarajan. "Internet Group Management Protocol, Version 3". RFC 3376. 2002.

- [14] R. Braden and D. Borman and C. Partridge. "Computing the Internet Checksum". RFC 1071. 1988.
- [15] "namespaces(7)". Linux man-pages. 2024.
- [16] "user_namespaces(7)". Linux man-pages. 2024.